A Largest-First Priority Queue for `mandelHybrid`'s Work Queue

Matias Saavedra

Monday 1st June, 2026

Binary: branch feat/priority-queue (intended tag binary-v4-pq) built on top of binary-v3-9point (ab47540). Baseline for the comparison is binary-v3-9point itself — the FIFO deque.

Resumen

WorkQueue's FIFO deque is replaced with a largest-first priority_queue (the existing MandelRegion::Compare, corrected to order by descending pixel count), in the hope — per 08-region-metrics-viz.tex's next-step 4 — of routing big regions, and in particular the ∼14 s interior outlier, to the fast GPU thread. Measured against the FIFO baseline on 100 frames at 1920 × 1080 (i7-9750H + GTX 1660 Ti Max-Q, 12 threads, diffT = 0.1, save = 0), the change is **wall-neutral**: 51.09 s vs. 51.37 s over 3 reps each (−0.55%, within noise). The Nsight Systems breakdown shows why nothing moves: **the GPU is equally utilised either way** (≈46.5 s busy, ∼91%), the **kernel max stays at ≈139 ms** in both — i.e. **the outlier is never routed to the GPU**; it still lands on a CPU thread (worst region 13.3 → 14.4 s) — and total compute is conserved (≈610 s). With eleven CPU workers versus one GPU thread, a *shared* largest-first queue hands big regions to whichever of the twelve extracts next, which is a CPU thread ∼11:1; and with the load already balanced under 2 % (report 09), reordering a work-conserving, throughput-bound schedule has nothing to gain. The report also documents a methodology near-miss: a battery-power run *falsely* showed a 2× "regression" because the GPU was power-capped to 30 W; an AC-power sanity gate caught it before any conclusion was drawn.

1. Setup and the Change Measured

Workload and machine.

Identical to reports 09: same spec.in (100 frames, 1920×1080, zoom to maxIterations = 10,000), i7-9750H (6c/12t) + GTX 1660 Ti Max-Q, 12 threads (1 GPU + 11 CPU), diffThreshold = 0.1, pixelSizeThresh = 32,768, save = 0. **On AC power** (see Section 5).

The change.

WorkQueue held a FIFO deque (push back, pop front). It is replaced by a std::priority_queue ordered by pixel count so extract() returns the largest pending region.

Código 1: workqueue.h / mandelregion.h — largest-first queue.

```
// workqueue.h
priority_queue<MandelRegion *, vector<MandelRegion *>, MandelRegion::Compare>
    ↪ queue;

// mandelregion.h -- Compare corrected to largest-first.
// priority_queue pops the element that is "greatest" under operator()(a,b)=="a
// before b"; returning a<b makes top() the region with the MOST pixels.
struct Compare {
  bool operator () (const MandelRegion *a, const MandelRegion *b) {
    return a->pixelsX * a->pixelsY < b->pixelsX * b->pixelsY;   // was '>': smallest-
      ↪ first
  }
};
```

The existing `Compare` (dead code until now) returned `>`, which in a `priority_queue` yields *smallest*-first; it was corrected to `<` for the intended largest-first (longest-processing-time) ordering. The decomposition is unchanged — the queue reorders *when* regions are processed, not which exist — so the leaf count stays exactly 5,608.

Measurement.

Four passes on both binaries: a wall-time A/B (3 reps, alternating); a `quiet` = 0 metrics run (leaves, per-thread totals, outliers); a Nsight Systems capture (NVTX ranges, CUDA API, CUPTI kernel); and a `quiet` = 0 distribution run. Baseline `binary-v3-9point` built in a worktree.

2. Results

2.1. Wall Time and Decomposition

Tabla 1: FIFO base vs. largest-first priority queue. A/B is the mean of 3 alternating reps (min–max). GPU/CPU split is scheduling-dependent; leaf total is deterministic.

Metric	FIFO (base)	Priority (largest-first)	Δ
Wall, A/B mean (s)	51.37	51.09	−0.28 (−0.55%)
A/B range (s)	51.21–51.52	50.96–51.33	overlapping
Total leaves	5,608	5,608	0
GPU-computed	4,194	4,014	−180
CPU-computed	1,414	1,594	+180
GPU kernel avg (ms)	10.74	11.41	+0.67
Worst CPU region (s)	13.3	14.4	+1.1
Outliers >5 s	1	1	0
CPU thread spread (s)	49.2–49.9	49.6–50.4	both <2%

2.2. GPU Utilisation and Kernel Tail (Nsight Systems)

Tabla 2: NVTX range totals (summed across threads) and CUPTI kernel statistics. The GPU-thread busy time and the kernel max are essentially identical — the priority queue neither starves nor feeds the GPU the big regions.

Quantity	FIFO (base)	Priority
`GPU region compute` total (s)	46.9	46.5
`CPU region compute` total (s)	569.0	558.6
`examine region` total (s)	620.2	609.0
Total compute (CPU+GPU, s)	615.9	605.1
`mandelKernel` instances	4,247	4,004
avg / max (ms)	11.0 / 139.0	11.6 / 139.0

2.3. Per-Region Distribution (`quiet` = 0)

Tabla 3: Per-region compute time (ms). The priority queue gives the GPU slightly larger regions and the CPU more, smaller ones, but the CPU tail (p99, max) is unchanged — the big regions stay on CPU.

Percentile	base CPU	prio CPU	base GPU	prio GPU
count	1,515	1,659	4,093	3,949
median	46.4	33.0	3.64	3.97
p99	4,003	4,092	—	—
max	13,494	14,896	139	139

3. Analysis

3.1. The wall does not move, and the GPU is not starved

The A/B delta is −0.55% with overlapping ranges (Table 1) — a wash. Table 2 rules out the two things one might fear: the GPU-thread busy time is ≈46.5–46.9 s in *both* (about 91% of the ∼51 s wall), so largest-first neither idles the GPU nor overloads it, and total compute is conserved (≈610 s, the same pixels). This is the report-09 regime: a throughput-bound schedule whose load is already balanced to under 2 % has no slack for a reordering to recover.

3.2. The outlier is never routed to the GPU — the premise fails

The motivating hope was that largest-first would hand the ∼14 s interior block to the GPU thread, where it would finish in milliseconds. It does not. The CUPTI kernel **max is ≈139 ms in both** binaries (Table 2): the GPU's biggest region is the same ∼139 ms region either way, so the 960×540 outlier was *not* dispatched to the GPU

under the priority queue. The CPU worst-region actually rose slightly (13.3 → 14.4 s, Table 1), and the CPU p99/max are unchanged (Table 3) — the big regions still run on CPU threads.

The mechanism is the *shared* queue. All twelve workers pull from one priority queue; when the outlier is enqueued, the next thread to call `extract()` takes it, and with eleven CPU threads against one GPU thread that is a CPU thread roughly 11:1. Largest-first changes the *order* regions are handed out, not *which processor* gets them. Routing big work to the GPU specifically would need GPU *affinity* (a separate GPU lane, or the GPU thread preferentially taking the largest while CPU threads take the smallest) — not a single shared largest-first queue.

3.3. What largest-first does change (but it does not matter here)

It does reshape the distribution as intended (Table 3): the GPU's regions get slightly larger (median 3.64 → 3.97 ms; avg 11.0 → 11.6 ms over fewer launches, 4247 → 4004) and the CPU gets more, smaller regions (count 1515 → 1659, median 46.4 → 33.0 ms). This is the longest-processing-time effect — big items first, small items filling the tail. It would help makespan on an *imbalanced* schedule; here the FIFO schedule was already balanced to under 2 %, so the reshaping is invisible at the wall.

3.4. Would the outlier even belong on the GPU? Yes — it is big *and* coherent

A natural objection to GPU affinity: these slow regions sit on the fractal boundary, so wouldn't they cause heavy warp divergence and run poorly on the GPU anyway? For *these* regions the answer is no, and the reason is the same property that makes them slow on the CPU.

They are big because they are interior, not because they are boundary.

The outliers are 960×540 (depth 1, a quarter of the frame) and 480×270 (depth 2); they stay large precisely because their corners are all interior and uniform, so the heuristic never splits them. The genuinely *divergent* regions — those straddling the boundary — have large corner spread and are therefore split all the way to the 240 × 135 pixel floor. Big-and-coherent and small-and-divergent; the pathological big-and-divergent case does not occur.

Warp divergence is low for an interior region.

A warp's runtime is set by its slowest (highest-iteration) thread; threads that escape early simply idle their lanes without adding work. Frame 73's region is 97.3 % interior, so in a 32-thread warp ∼31 threads run to `maxIter` together — only ∼3 % of lanes are wasted. The cost is the `maxIter` arithmetic itself (which the GPU does in parallel), not divergence. "97.3 % interior" here means 97.3 % of that region's pixels reach `maxIter` (in the set — specifically inside a *minibrot*, since `cardioidBulb` = 0, not the main cardioid/bulb).

Direct evidence: same-size region, both processors.

Tabla 4: Per-region cost by processor (clean run). A coherent interior 480 × 270 region costs ∼29× less on the GPU than on a CPU thread; the divergent boundary regions are split to 240×135 and are cheap on the GPU regardless.

Region	type	GPU kernel	CPU compute
240 × 135 (depth 3, floor)	boundary/mixed	≤36 ms	<1 s
480 × 270 (depth 2)	interior	139 ms (max)	∼4 s (CPU p99)
960 × 540 (depth 1)	interior	∼0.45 s (est., 4×)	13.3 s (the outlier)

The GPU never received a 960 × 540 region (the shared queue sent them to CPU ∼11:1), but it did process 183 interior 480 × 270 regions, the largest at 139 ms — versus ∼4 s for the same size on a CPU thread (∼29×). The outlier, at 4× the pixels, would be ∼0.45 s on the GPU versus 13.3 s on a CPU thread. So routing it to the GPU is a ∼30× win, not a divergence trap.

Caveat: the kernel is double precision.

`diverge()` iterates in `double`, and this GPU (Turing TU116) executes FP64 at 1:32 of FP32 — a ∼160 GFLOPS ceiling versus its ∼5 TFLOPS FP32 peak. Even the coherent 480 × 270 region reaches only ∼40 % of that FP64 ceiling. Recasting the iteration in `float` would be far faster on the GPU, but FP32's ∼7 significant digits cannot resolve the deepest frames (coordinates to ∼15 digits), so it would corrupt the late zoom; double is required for correctness, and the GPU is still ∼30× faster than the serial CPU on these regions. This reinforces recommendation 2: the outlier is the ideal GPU candidate — large, coherent, ∼30× faster — and the shared priority queue's only failure was the 11:1 extraction that kept it on the CPU.

4. Methodology Note: Power-State Contamination and the AC Gate

This experiment was nearly reported with the opposite conclusion. An initial run showed the priority queue at ∼104 s and a later full pass put *both* binaries at ∼147 s — an apparent 2× regression. The cause was not the code: the laptop was on **battery**, and the GTX 1660 Ti Max-Q had its power limit clamped to **30 W** (default 60 W, `SW Power Cap` active), so the GPU drew 14 W at 1140 MHz, processed far fewer regions (GPU leaves ≈1,670 vs. the healthy ≈4,190), and the CPU pool absorbed the rest. The CPU was also on a power-saving profile.

Two lessons. First, **never compare a fresh run against historical numbers without confirming both ran in the same power state** — the false 2× "regression" came from comparing a battery run against report-09's AC numbers. Second, an **AC-power sanity gate** now precedes any timing pass: one FIFO base run must return ≈52 s with GPU ≥ 4,000 leaves, or the pass is aborted. After a reboot restored the 60 W limit, the gate passed (50.5 s, GPU 4,167, GPU drawing 35.8 W at 1845 MHz mid-run) and the numbers above were collected. All measurements in this report are post-gate.

5. Recommendations

1. **Do not adopt the shared largest-first queue as the default.** It is wall-neutral (−0.55%, within noise), decomposition-identical, and adds `priority_queue` bookkeeping for no measured benefit. Keep the FIFO `deque` for simplicity. (If kept, it is harmless — correctness is unchanged.)

2. **If the goal is to offload the outlier to the GPU, use GPU affinity, not a shared queue.** Have the GPU thread preferentially pull the largest region (or a dedicated large-region lane) while CPU threads pull smaller ones. The 960×540 block costs ∼14 s on a CPU thread but only ∼139 ms on the GPU; routing *it* specifically is the only version of this idea with real upside — it would cut the worst-region tail by two orders of magnitude. A shared largest-first queue cannot do this (11:1 CPU:GPU extraction).

3. **The binding lever remains work-reduction (report 09).** Even a perfect schedule cannot beat the conserved ∼610 s of compute; periodicity checking in `diverge()` or an interior certificate is what shrinks it.

6. Conclusion

Replacing the FIFO work queue with a largest-first priority queue is wall-neutral on this hybrid (−0.55%, within noise). The Nsight Systems breakdown shows the GPU is equally utilised either way and, decisively, that the kernel max is unchanged at ≈139 ms — the ∼14 s outlier is never routed to the GPU, because a single shared queue hands big regions to whichever of the twelve workers extracts next, a CPU thread ∼11:1. With the load already balanced under 2 % and total compute conserved, reordering has nothing to recover. The lever that would help — routing the outlier to the ∼40×-faster GPU — requires GPU *affinity*, not a shared priority queue; and the lever that bounds the wall remains work-reduction. The experiment also reaffirmed a measurement discipline: a battery-induced 30 W GPU cap had falsely shown a 2× regression, which an AC-power gate caught before it reached a conclusion.